1) Python program to display:

```
print("Hello World.")
print("This is Python.")
```

2) Python program to work with data types

```
# Integer
a = 10
print("Integer:", a, "Type:", type(a))
# Float
b = 3.14
print("Float:", b, "Type:", type(b))
# Complex
c = 2 + 3j
print("Complex:", c, "Type:", type(c))
# Boolean
d = True
print("Boolean:", d, "Type:", type(d))
# String
e = "Hello Python"
print("String:", e, "Type:", type(e))
```

(3) Built-in Functions

```
a = 10
print("ID of a:", id(a))
print("Type of a:", type(a))
print("Range from 0 to 4:", list(range(5)))
```

(4) Type Conversion

```
x = "100"
y = int(x)
print("Converted to int:", y)
print("Converted to float:", float(x))
print("Converted to string:", str(y))
```

(5) Operators in Python

```
a = 10
b = 3
print("Addition:", a + b)
print("Is a > b?", a > b)
a += 5
print("After += :", a)
print("Logical AND:", True and False)
print("Bitwise AND:", a & b)
max_value = a if a > b else b
print("Max value:", max_value)
```

(6) Input & Print

```
name = input("Enter your name: ")
print("Welcome", name)
print("Python", "is", "fun", sep="-", end="!")
print("\n")
age = 20
print(f"{name} is {age} years old.")
```

(7) Conditional Statements

```
num = int(input("Enter a number: "))
if num > 0:
   print("Positive")
```

```
elif num == 0:
   print("Zero")
else:
   print("Negative")
```

(8) Iterative Statements

```
i = 1
while i <= 5:
   print(i)
   i += 1

for i in range(5):
   print("Iteration", i)
```

(9) Control Transfer

```
for i in range(5):
   if i == 2:
      continue
   if i == 4:
      break
   print(i)
else:
   pass
```

(10) String Indexing and Slicing

```
s = "Python"
print("First char:", s[0])
print("Last char:", s[-1])
print("First 3 chars:", s[:3])
```

(11) File Read and Write

```
with open("test.txt", "w") as f:
   f.write("Hello Students!")
with open("test.txt", "r") as f:
   content = f.read()
   print("File content:", content)
```

(12) Copy File Content

```
with open("source.txt", "r") as src, open("copy.txt", "w") as dst:
    dst.write(src.read())
```

(13) Frequency of Characters

```
text = "banana"
freq = {}
for char in text:
    freq[char] = freq.get(char, 0) + 1
print(freq)
```

(14) Print Each Line

```
with open("test.txt", "r") as file:
    for line in file:
        print(line.strip())
```

(15) Count Characters, Words, Lines

```
with open("test.txt", "r") as file:
    text = file.read()
chars = len(text)
words = len(text.split())
lines = len(text.splitlines())
print("Characters:", chars)
print("Words:", words)
print("Lines:", lines)
```

(16) Python program to create list objects in different ways

```
# Empty list

list1 = []

# List with values

list2 = [1, 2, 3, 4, 5]

# List using list() function

list3 = list("hello")

# List using range

list4 = list(range(1, 6))

print("Empty List:", list1)

print("List with values:", list2)

print("List from string:", list3)

print("List using range:", list4)
```

(17) Python program demonstrating list functions/methods

```
lst = [10, 20, 30, 40]

print("Original List:", lst)

print("Length:", len(lst))

print("Count of 20:", lst.count(20))

print("Index of 30:", lst.index(30))

lst.append(50)

print("After append(50):", lst)

lst.insert(2, 25)

print("After insert at index 2:", lst)

lst.extend([60, 70])

print("After extend:", lst)

lst.remove(25)

print("After remove 25:", lst)

lst.pop()

print("After pop():", lst)

lst.reverse()

print("After reverse():", lst)

lst.sort()

print("After sort():", lst)

copy_list = lst.copy()

print("Copied List:", copy_list)

lst.clear()
```

```
print("After clear():", lst)
```

(18) Python program to create tuple objects in different ways

```
# Empty tuple

t1 = ()

# Tuple with values

t2 = (1, 2, 3)

# Tuple from list

t3 = tuple([4, 5, 6])

# Tuple with mixed data types

t4 = (10, "hello", 3.14)

print("Empty tuple:", t1)

print("Tuple with values:", t2)

print("Tuple from list:", t3)

print("Mixed tuple:", t4)
```

(19) Python programs to print patterns

```
1

22

333

4444

55555

for i in range(1, 6):

   print(str(i) * i)
```

```
A
A B
A B C
A B C D
A B C D E
for i in range(1, 6):
    for j in range(65, 65+i):
        print(chr(j), end=" ")
    print()
*****
****
***
**
*
for i in range(5, 0, -1):
    print("*" * i)
```

(20) Python program demonstrating tuple functions/methods

```
t = (5, 3, 8, 6, 3)

print("Tuple:", t)

print("Length:", len(t))

print("Count of 3:", t.count(3))

print("Index of 8:", t.index(8))

print("Sorted:", sorted(t))

print("Min:", min(t))

print("Max:", max(t))

# Python does not have cmp(), so we use comparison operators

t2 = (5, 3, 8)

print("Comparison with (5, 3, 8):", t == t2)

print("Reversed:", tuple(reversed(t)))
```

(21) Create Set Objects in Different Ways

```
# Empty set

set1 = set()

# Set with values

set2 = {1, 2, 3, 4}

# Set from list

set3 = set([5, 6, 7])

# Set with mixed types

set4 = {1, "Python", 3.14}

print("Empty set:", set1)
```

```
print("Set with values:", set2)
print("Set from list:", set3)
print("Mixed set:", set4)
```

(22) Set Functions/Methods Example

```
s = {10, 20, 30}
print("Original set:", s)
s.add(40)
print("After add(40):", s)
s.update([50, 60])
print("After update([50, 60]):", s)
s2 = s.copy()
print("Copied set:", s2)
s.pop()
print("After pop():", s)
s.discard(20)
print("After discard(20):", s)
s.remove(30)
print("After remove(30):", s)
s.clear()
print("After clear():", s)

# Set operations
a = {1, 2, 3}
```

```
b = {3, 4, 5}


print("Set A:", a)
print("Set B:", b)
print("Union:", a.union(b))
print("Intersection:", a.intersection(b))
print("Difference:", a.difference(b))
```

(23) Create Dictionary Objects in Different Ways

```
# Empty dictionary
dict1 = {}
# Dictionary with values
dict2 = {'name': 'Alice', 'age': 25}
# Dictionary using dict() constructor
dict3 = dict([('x', 1), ('y', 2)])
# Dictionary with mixed keys
dict4 = {1: 'One', 'Two': 2}
print("Empty dictionary:", dict1)
print("With values:", dict2)
print("Using dict():", dict3)
print("Mixed keys:", dict4)
```

(24) Dictionary Functions/Methods Example

```
d = {'name': 'Bob', 'age': 22, 'city': 'Rajkot'}
```

```
print("Original dictionary:", d)
print("Length:", len(d))
print("Get 'age':", d.get('age'))
d.update({'gender': 'Male'})
print("After update:", d)
d.pop('city')
print("After pop('city'):", d)
d['country'] = 'India'
print("After adding key 'country':", d)
print("Keys:", d.keys())
print("Values:", d.values())
print("Items:", d.items())
d2 = d.copy()
print("Copied dictionary:", d2)
d.clear()
print("After clear:", d)
```

(25) Return Multiple Values from Function

```
def student_details():
    name = "Yash"
    age = 21
    course = "BCA"
    return name, age, course
# Receiving multiple values
```

```
n, a, c = student_details()
print("Name:", n)
print("Age:", a)
print("Course:", c)
```

(26) Local and Global Variables Example

```
x = 10  # Global variable
def show():
    x = 5  # Local variable
    print("Local x:", x)
def display():
    global x
    x = x + 5
    print("Modified Global x:", x)
show()
display()
print("Global x after function call:", x)
```

(27) Lambda Function Example

```
square = lambda x: x * x
print("Square of 5:", square(5))
add = lambda a, b: a + b
print("Sum of 3 and 4:", add(3, 4))
```

(28) Turn 60 Years Old Message

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
```

```
year = 2025 + (60 - age)
print(f"Hello {name}, you will turn 60 in the year {year}.")
```

(29) Even or Odd Message

```
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("The number is Even.")
else:
    print("The number is Odd.")
```

(30) Fibonacci Series Generator

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        print(a, end=" ")
        a, b = b, a + b
fibonacci(10)
```

(31) Reverse a String

```
def reverse_string(s):
```

```
    return s[::-1]

print("Reversed:", reverse_string("hello"))
```

(32) Armstrong Number Check

```
num = int(input("Enter a number: "))
sum = 0
temp = num
while temp > 0:
   digit = temp % 10
   sum += digit ** 3
   temp //= 10
if num == sum:
   print(num, "is an Armstrong number.")
else:
   print(num, "is not an Armstrong number.")
```

(33) Palindrome Check

```
def is_palindrome(s):
   return s == s[::-1]
print(is_palindrome("madam"))
```

(33) Recursive Factorial

```
def factorial(n):
   if n == 0 or n == 1:
      return 1
   else:
      return n * factorial(n-1)
print("Factorial of 5:", factorial(5))
```

(34) Check for Vowel

```
def is_vowel(char):
    return char.lower() in 'aeiou'


print(is_vowel('a'))  # True
print(is_vowel('b'))  # False
```

(35) Length of List or String

```
def compute_length(item):
    return len(item)


print("Length of list:", compute_length([1, 2, 3, 4]))
print("Length of string:", compute_length("hello"))
```

(36) Remove 0th, 2nd, 3rd, 5th Elements

What is enumerate(lst) in Python?

enumerate() is a built-in Python function that adds a counter (index) to an iterable (like a list, string, or tuple). It returns an enumerate object, which can be used in loops.

```
def remove_indices(lst):
    indices = [0, 2, 3, 5]
        return [item for i, item in enumerate(lst) if i not in indices]


print(remove_indices([10, 20, 30, 40, 50, 60, 70]))
```

(37) Sort Dictionary by Value

```
d = {'a': 3, 'b': 1, 'c': 2}

# Ascending

asc = dict(sorted(d.items(), key=lambda item: item[1]))

print("Ascending:", asc)

# Descending

desc = dict(sorted(d.items(), key=lambda item: item[1],
reverse=True))

print("Descending:", desc)
```

(38) Sum of Items in Dictionary

```
d = {'a': 10, 'b': 20, 'c': 30}

print("Sum of values:", sum(d.values()))
```

(39) Searching and Sorting Techniques

i) Linear Search

```
def linear_search(lst, target):
    for i in range(len(lst)):
        if lst[i] == target:
            return i
    return -1
```

ii) Binary Search (Requires sorted list)

```
def binary_search(lst, target):
    low, high = 0, len(lst)-1
    while low <= high:
        mid = (low + high) // 2
        if lst[mid] == target:
            return mid
        elif lst[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

iii) Selection Sort

```
def selection_sort(lst):
    for i in range(len(lst)):
        min_idx = i
        for j in range(i+1, len(lst)):
            if lst[j] < lst[min_idx]:
                min_idx = j
        lst[i], lst[min_idx] = lst[min_idx], lst[i]
    return lst
```

iv) Bubble Sort

```
def bubble_sort(lst):
    n = len(lst)
    for i in range(n):
        for j in range(0, n-i-1):
            if lst[j] > lst[j+1]:
                lst[j], lst[j+1] = lst[j+1], lst[j]
    return lst
```

v) Insertion Sort

```
def insertion_sort(lst):
    for i in range(1, len(lst)):
        key = lst[i]
        j = i - 1
        while j >= 0 and key < lst[j]:
            lst[j+1] = lst[j]
            j -= 1
        lst[j+1] = key
    return lst
```

vi) Merge Sort

```
def merge_sort(lst):
    if len(lst) > 1:
```

```
mid = len(lst)//2
L = lst[:mid]
R = lst[mid:]

merge_sort(L)
merge_sort(R)

i = j = k = 0
while i < len(L) and j < len(R):
    if L[i] < R[j]:
        lst[k] = L[i]
        i += 1
    else:
        lst[k] = R[j]
        j += 1
    k += 1
while i < len(L):
    lst[k] = L[i]
    i += 1
    k += 1
while j < len(R):
    lst[k] = R[j]
    j += 1
    k += 1
```

return lst

vii) Quick Sort

```
def quick_sort(lst):
    if len(lst) <= 1:
        return lst
    else:
        pivot = lst[0]
        left = [x for x in lst[1:] if x <= pivot]
        right = [x for x in lst[1:] if x > pivot]
        return quick_sort(left) + [pivot] + quick_sort(right)
```

(40) Encapsulation Concept

```
class Student:
    def __init__(self, name, age):
        self.__name = name  # private
        self.__age = age    # private

    def show(self):
        print(f"Name: {self.__name}, Age: {self.__age}")

    def set_age(self, age):
        if age > 0:
            self.__age = age

s = Student("Yash", 21)
s.show()
```

s.set_age(22)

s.show()

Python Programs: 41 - 63

(41) Generate different plotting using PyLab. Such as Line plot, Bar chart, Pie chart, Histogram, Scatter plot

```
import matplotlib.pylab as plt

import numpy as np

# Sample data

x = np.linspace(0, 10, 10)

y = np.sin(x)

y2 = np.cos(x)

values = [5, 7, 3, 4, 6]

# Line Plot

plt.figure(figsize=(10, 6))

plt.subplot(2, 3, 1)

plt.plot(x, y, label="sin(x)", marker="o")

plt.plot(x, y2, label="cos(x)", marker="s")

plt.title("Line Plot")

plt.legend()

# Bar Chart

plt.subplot(2, 3, 2)

plt.bar(range(len(values)), values, color="skyblue")

plt.title("Bar Chart")

# Pie Chart

plt.subplot(2, 3, 3)

sizes = [20, 30, 25, 15, 10]

labels = ["A", "B", "C", "D", "E"]

plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
```

```
plt.title("Pie Chart")

# Histogram

plt.subplot(2, 3, 4)

data = np.random.randn(1000)  # 1000 random numbers

plt.hist(data, bins=20, color="lightgreen", edgecolor="black")

plt.title("Histogram")

# Scatter Plot

plt.subplot(2, 3, 5)

x_scatter = np.random.rand(50)

y_scatter = np.random.rand(50)

plt.scatter(x_scatter, y_scatter, color="red", marker="x")

plt.title("Scatter Plot")

plt.tight_layout()

plt.show()
```

(42) Plotting a curve (Line plot example)

```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x)

plt.plot(x, y, label='Sine Curve')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Line Plot Example')
plt.legend()
plt.show()
```

(43) 0/1 Knapsack (Dynamic Programming)

```
def knapsack(values, weights, W):
    n = len(values)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i-1][w], dp[i-1][w-weights[i-1]] +
```

```
values[i-1])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]

# Example
vals = [60, 100, 120]
wts = [10, 20, 30]
W = 50
print(knapsack(vals, wts, W))
```

(44) Divide and Conquer (Merge Sort)

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(a, b):
    i = j = 0
    res = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            res.append(a[i]); i += 1
        else:
            res.append(b[j]); j += 1
    res.extend(a[i:]); res.extend(b[j:])
    return res

print(merge_sort([5,2,9,1,5,6]))
```

(45) Create a socket (TCP client example)

```
import socket

HOST = '127.0.0.1'
PORT = 65432
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b'Hello, server')
    data = s.recv(1024)
print('Received', repr(data))
```

(46) Identify IP address (local)

```
import socket
hostname = socket.gethostname()
local_ip = socket.gethostbyname(hostname)
print('Hostname:', hostname)
print('Local IP:', local_ip)
```

(47) Download source code of a web page (requests)

```
import requests
url = 'https://example.com'
r = requests.get(url)
with open('page_source.html', 'w', encoding='utf-8') as f:
    f.write(r.text)
```

(48) Download a web page from internet (urllib)

```
from urllib.request import urlopen
url = 'https://example.com'
resp = urlopen(url)
html = resp.read().decode('utf-8')
with open('downloaded_page.html', 'w', encoding='utf-8') as f:
    f.write(html)
```

(49) Download an image from internet

```
import requests
url = 'https://www.example.com/image.jpg'
r = requests.get(url)
with open('image.jpg', 'wb') as f:
    f.write(r.content)
```

(50) Create a TCP/IP Server and client (simple)

```
# Server (server.py)
import socket
HOST = '127.0.0.1'
PORT = 65432
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    conn, addr = s.accept()
    with conn:
        print('Connected by', addr)
        while True:
```

```
            data = conn.recv(1024)
            if not data: break
            conn.sendall(data.upper())

# Client (client.py)
import socket
HOST = '127.0.0.1'
PORT = 65432
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b'hello server')
    data = s.recv(1024)
print('Received', data)
```

(51) Two-way communication (simple echo)

```
# Server handles messages and can send prompts back; it's similar to
50.
# Use threading for multiple clients.
import socket, threading

def handle(conn, addr):
    with conn:
        print('Connected', addr)
        while True:
            data = conn.recv(1024)
            if not data: break
            print('Client says:', data.decode())
            reply = input('Reply> ').encode()
            conn.sendall(reply)

s = socket.socket()
s.bind(('127.0.0.1', 9000))
s.listen()
while True:
    conn, addr = s.accept()
    threading.Thread(target=handle, args=(conn, addr),
daemon=True).start()
```

(52) UDP server and client

```
# UDP Server
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(('127.0.0.1', 9999))
while True:
    data, addr = s.recvfrom(1024)
```

```
    print('Received', data, 'from', addr)
    s.sendto(b'ACK', addr)

# UDP Client
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.sendto(b'Hello UDP', ('127.0.0.1', 9999))
data, addr = s.recvfrom(1024)
print('Response', data)
```

(53) File server and client (send file over TCP)

```
# Server
import socket
s = socket.socket()
s.bind(('0.0.0.0', 6000))
s.listen(1)
conn, addr = s.accept()
with conn, open('sendfile.bin','rb') as f:
    for chunk in iter(lambda: f.read(4096), b''):
        conn.sendall(chunk)
# Client
import socket
s = socket.socket()
s.connect(('127.0.0.1', 6000))
with open('received.bin','wb') as f:
    while True:
        data = s.recv(4096)
        if not data: break
        f.write(data)
```

(54) Sending an email (smtplib)

```
import smtplib
from email.message import EmailMessage

msg = EmailMessage()
msg['Subject'] = 'Test mail'
msg['From'] = 'you@example.com'
msg['To'] = 'recipient@example.com'
msg.set_content('Hello from Python')

with smtplib.SMTP('smtp.example.com', 587) as s:
    s.starttls()
    s.login('username', 'password')
    s.send_message(msg)
```

(55) GUI with buttons, labels, entry fields (Tkinter)

```
import tkinter as tk

root = tk.Tk()
tk.Label(root, text='Name').pack()
e = tk.Entry(root); e.pack()
tk.Button(root, text='Click', command=lambda: print(e.get())).pack()
root.mainloop()
```

(56) Dialogs (confirmation, alerts)

```
import tkinter as tk
from tkinter import messagebox as mb

root = tk.Tk()
def on_quit():
    if mb.askyesno('Verify','Really quit?'):
        mb.showinfo('Quit','Exiting now')
        root.destroy()

tk.Button(root, text='Quit', command=on_quit).pack()
root.mainloop()
```

(57) Simple calculator (Tkinter)

```
import tkinter as tk

def calculate():
    try:
        res = eval(entry.get())
        result_var.set(str(res))
    except Exception as e:
        result_var.set('Err')

root = tk.Tk()
entry = tk.Entry(root); entry.pack()
tk.Button(root, text='=', command=calculate).pack()
result_var = tk.StringVar(); tk.Label(root,
textvariable=result_var).pack()
root.mainloop()
```

(58) List existing databases (MySQL example using mysql-connector)

```
import mysql.connector

cnx = mysql.connector.connect(user='root', password='pwd',
host='127.0.0.1')
cursor = cnx.cursor()
cursor.execute('SHOW DATABASES')
for db in cursor:
    print(db)
```

(59) Insert a row into a table (MySQL)

```
import mysql.connector
cnx = mysql.connector.connect(user='root', password='pwd',
host='127.0.0.1', database='testdb')
cur = cnx.cursor()
cur.execute('INSERT INTO tbl (name) VALUES (%s)', ('Alice',))
cnx.commit()
cur.close(); cnx.close()
```

(60) Update a row into a table (MySQL)

```
import mysql.connector
cnx = mysql.connector.connect(user='root', password='pwd',
host='127.0.0.1', database='testdb')
cur = cnx.cursor()
cur.execute('UPDATE tbl SET name=%s WHERE id=%s', ('Bob', 1))
cnx.commit()
cur.close(); cnx.close()
```

(61) Create dbStudent and insert student info

```
# Create DB and table using MySQL client or python connector.
# Insert example:
import mysql.connector
cnx = mysql.connector.connect(user='root', password='pwd',
host='127.0.0.1')
cur = cnx.cursor()
cur.execute('CREATE DATABASE IF NOT EXISTS dbStudent')
cur.execute('USE dbStudent')
cur.execute('''CREATE TABLE IF NOT EXISTS tblStudInfo (
    student_id INT PRIMARY KEY AUTO_INCREMENT,
    student_name VARCHAR(100),
    stream VARCHAR(50),
    college_name VARCHAR(150),
    contact_number VARCHAR(30),
```

```
    remarks TEXT)''')
cur.execute('INSERT INTO tblStudInfo (student_name, stream,
college_name, contact_number, remarks) VALUES (%s,%s,%s,%s,%s)',
            ('John Doe','CS','ABC College','9999999999','Good'))
cnx.commit()
cur.close(); cnx.close()
```

(62) Update student information

```
import mysql.connector
cnx = mysql.connector.connect(user='root', password='pwd',
host='127.0.0.1', database='dbStudent')
cur = cnx.cursor()
cur.execute('UPDATE tblStudInfo SET remarks=%s WHERE student_id=%s',
('Excellent', 1))
cnx.commit()
cur.close(); cnx.close()
```

(63) Delete student information

```
import mysql.connector
cnx = mysql.connector.connect(user='root', password='pwd',
host='127.0.0.1', database='dbStudent')
cur = cnx.cursor()
cur.execute('DELETE FROM tblStudInfo WHERE student_id=%s', (1,))
cnx.commit()
cur.close(); cnx.close()
```

Form Using Tkinter

```
from tkinter import *


def submit():
    fname = entry_fname.get()
    lname = entry_lname.get()
    gender = "Male" if var_gender.get() == 1 else "Female"
```

```
    subjects = [listbox.get(i) for i in listbox.curselection()]
    print("First Name:", fname)
    print("Last Name:", lname)
    print("Gender:", gender)
    print("Subjects:", subjects)

master = Tk()
master.title("Student Form")
master.geometry("400x300")

# --- Frame for personal info ---
frame1 = Frame(master, padx=10, pady=10, relief=RIDGE, borderwidth=2)
frame1.pack(padx=10, pady=10, fill=X)

Label(frame1, text="First Name:").grid(row=0, column=0, sticky=W, pady=2)
entry_fname = Entry(frame1)
entry_fname.grid(row=0, column=1, pady=2)

Label(frame1, text="Last Name:").grid(row=1, column=0, sticky=W, pady=2)
entry_lname = Entry(frame1)
entry_lname.grid(row=1, column=1, pady=2)

# --- Frame for gender selection ---
frame2 = Frame(master, padx=10, pady=10, relief=RIDGE, borderwidth=2)
frame2.pack(padx=10, pady=5, fill=X)
```

```
Label(frame2, text="Gender:").grid(row=0, column=0, sticky=W)

var_gender = IntVar()

Radiobutton(frame2, text="Male", variable=var_gender, value=1).grid(row=0, column=1)

Radiobutton(frame2, text="Female", variable=var_gender, value=2).grid(row=0,
column=2)


# --- Frame for subjects list ---

frame3 = Frame(master, padx=10, pady=10, relief=RIDGE, borderwidth=2)

frame3.pack(padx=10, pady=5, fill=X)


Label(frame3, text="Subjects:").pack(anchor=W)

listbox = Listbox(frame3, selectmode=MULTIPLE, height=4)

listbox.pack(fill=X)

for subject in ["Python", "J2EE", "CS"]:

    listbox.insert(END, subject)


# --- Submit Button ---

Button(master, text="Submit", command=submit, fg="white", bg="green").pack(pady=10)


master.mainloop()
```